

Reviewer Pack — Symmetric Polynomial Monomial Count and Resolution Proof Siz...

Entry 5421dda86f76 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 01:10:48 UTC

Symmetric Polynomial Monomial Count and Resolution Proof Size for Tseitin Formulas

Entry ID: 5421dda86f76 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 01:10:48 UTC

1. Conjecture

Field A (mathematical branch): Symmetric Polynomials (Algebraic Combinatorics) **Field B** (complexity object): Resolution Proof Size (Tseitin Formulas)

Statement:

For a Tseitin formula derived from a bipartite graph with n edges, the size of the minimal resolution proof is $\Theta(m)$, where m is the number of distinct monomials in the symmetric polynomial generated by the graph's adjacency matrix.

Rationale (proposer's reasoning):

The symmetric polynomial's monomial count reflects the graph's structural complexity, which may impose constraints on the resolution steps required to derive a contradiction, leading to a proportional increase in proof size.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6eeabe09a533f483

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	HITS	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def generate_primes(count):
    primes = []
    num = 2
    while len(primes) < count:
        if is_prime(num):
            primes.append(num)
        num += 1
    return primes

def adjacency_matrix(A, B):
    n = len(A)
    m = len(B)
    adj = [[0] * (n + m) for _ in range(n + m)]
    for i in range(n):
        for j in range(m):
            if A[i][j]:
                adj[i][len(A) + j] = 1
                adj[len(A) + j][i] = 1
    return adj

def gaussian_elimination(matrix, b):
    n = len(matrix)
    augmented_matrix = [row[:] + [b[i]] for i, row in enumerate(matrix)]

    for i in range(n):
        max_row = i
        for j in range(i + 1, n):
            if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
```

```

        max_row = j

    augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]

    factor = augmented_matrix[i][i]
    for j in range(i, n + 1):
        augmented_matrix[i][j] /= factor

    for j in range(n):
        if j != i:
            factor = augmented_matrix[j][i]
            for k in range(i, n + 1):
                augmented_matrix[j][k] -= factor * augmented_matrix[i][k]

x = [0] * n
for i in range(n - 1, -1, -1):
    x[i] = augmented_matrix[i][-1]
    for j in range(i + 1, n):
        x[i] -= augmented_matrix[i][j] * x[j]

return x

def tseitin_formula(A, B):
    n = len(A)
    m = len(B)
    clauses = []
    literals = {}
    var_count = 0

    for i in range(n):
        literals[(i, 'A')] = var_count
        var_count += 1

    for j in range(m):
        literals[(j, 'B')] = var_count
        var_count += 1

    for i in range(n):
        clauses.append([literals[(i, 'A')]])

    for j in range(m):
        clauses.append([-literals[(j, 'B')]])

    for i in range(n):
        for j in range(m):
            if A[i][j]:
                clauses.append([-literals[(i, 'A')], literals[(j, 'B')]])
                clauses.append([literals[(i, 'A')], -literals[(j, 'B')]])

    return clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 30
    m = random.randint(2 * n // 3, n)

```

```

A = [[random.choice([0, 1]) for _ in range(m)] for _ in range(n)]
B = [[random.choice([0, 1]) for _ in range(n)] for _ in range(m)]

adj = adjacency_matrix(A, B)
monomials = set()

def dfs(node, path):
    if node == len(A) + m:
        monomials.add(tuple(sorted(path)))
        return
    if node < len(A):
        dfs(node + 1, path + [node])
    else:
        for j in range(m):
            if A[node - len(A)][j]:
                dfs(j, path)

dfs(0, [])

clauses = tseitin_formula(A, B)
num_clauses = len(clauses)

def dpll(clauses, assignment):
    if not clauses:
        return True
    unit_clause = next((c for c in clauses if len(c) == 1), None)
    if unit_clause:
        literal = unit_clause[0]
        new_assignment = assignment[:]
        new_assignment[abs(literal) - 1] = literal > 0
        return dpll(clauses, new_assignment)

    literal = next((c[0] for c in clauses if not any(abs(x) == abs(c[0]) for x in assignment)), None)
    if literal is None:
        return False

    new_clauses_true = [c for c in clauses if literal not in c and -literal not in c]
    new_assignment_true = assignment[:]
    new_assignment_true[abs(literal) - 1] = literal > 0
    if dpll(new_clauses_true, new_assignment_true):
        return True

    new_clauses_false = [c for c in clauses if -literal not in c and literal not in c]
    new_assignment_false = assignment[:]
    new_assignment_false[abs(literal) - 1] = literal < 0
    if dpll(new_clauses_false, new_assignment_false):
        return True

    return False

proof_size = 0
for _ in range(10): # Sample multiple instances per seed
    assignment = [False] * num_clauses
    if not dpll(clauses, assignment):
        proof_size += 1

```

```

metric_value = len(monomials)
conjecture_holds = proof_size == metric_value
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "resolution_proof_size",
    "metric_value": proof_size,
    "instances_tested": 10,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or generate_primes(30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{result}}}"))
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ome/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_elfee859.py", line 120, in dfs
    dfs(node + 1, path + [node])
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_elfee859.py", line 120, in dfs
    dfs(node + 1, path + [node])
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_elfee859.py", line 120, in dfs
    dfs(node + 1, path + [node])
[Previous line repeated 27 more times]
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_elfee859.py", line 124, in dfs
    dfs(j, path)
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_elfee859.py", line 120, in dfs
    dfs(node + 1, path + [node])
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_elfee859.py", line 120, in dfs
    dfs(node + 1, path + [node])
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_elfee859.py", line 120, in dfs
    dfs(node + 1, path + [node])
[Previous line repeated 27 more times]
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_elfee859.py", line 124, in dfs

```

```

dfs(j, path)
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_elfee859.py", line 120, in dfs
dfs(node + 1, path + [node])
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_elfee859.py", line 120, in dfs
dfs(node + 1, path + [node])
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_elfee859.py", line 120, in dfs
dfs(node + 1, path + [node])
[Previous line repeated 27 more times]
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_elfee859.py", line 124, in dfs
dfs(j, path)
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_elfee859.py", line 120, in dfs
dfs(node + 1, path + [node])
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_elfee859.py", line 120, in dfs
dfs(node + 1, path + [node])
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_elfee859.py", line 120, in dfs
dfs(node + 1, path + [node])
[Previous line repeated 27 more times]
File "/home/ludo/

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed during execution, preventing data collection to verify conjecture | next: Debug the recursive DFS implementation to identify stack overflow cause

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109929
2	propose	ollama_remote	qwen3:8b	0	0	97119
3	preregistration	ollama_remote	qwen3:8b	0	0	23966
4	novelty	ollama_remote	qwen3:8b	0	0	20544
5	novelty	ollama_remote	qwen3:8b	0	0	18774
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11768
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17939
8	judge	ollama_remote	qwen3:8b	0	0	16321

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 316361 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/5421dda86f76.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5421dda86f76.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5421dda86f76.tar.gz` (if generated)